

표적 궤적 모의

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그리기까지

개념편 · 코드 구현은 다음 발표에서

레이다 시스템 소프트웨어 스터디 · Part 3

INDEX

목차

00 들어가며 과제 3이 묻는 것
과제 2와 무엇이 다른가

01 표적을 움직이는 원리 좌표계 넷 · 위치와 속도
시간 적분 · 지구 곡률

02 기동 모델 하중배수 g · 원운동
세 축 · 기동 시간표

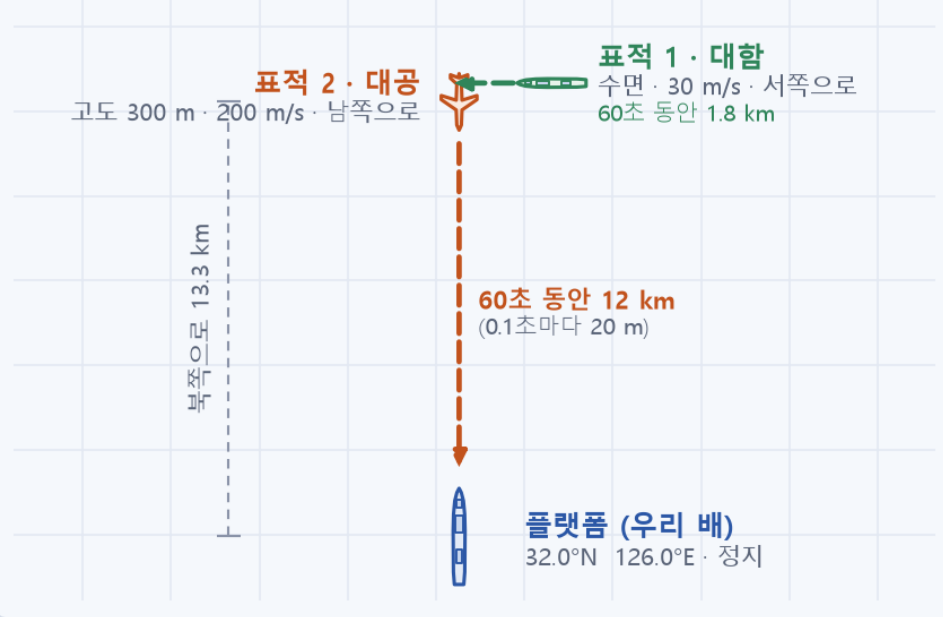
03 구조체와 시뮬레이션 설계 표적 카드 · 전체 흐름 · 조건
예상 결과 · 검증 · 다음 단계

오늘은 개념편입니다. 수학은 고등학교 물리의 등속 원운동 정도면 충분하고, 나올 때마다 다시 설명합니다.

00. 과제 3이 묻는 것

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라

위에서 내려다본 그림 (대략 축척)



시뮬레이션 시간

60 초 · 0.1 초 간격

- $t = 0.0 \text{ s}$
- $t = 0.1 \text{ s}$
- $t = 0.2 \text{ s}$
- $t = 0.3 \text{ s}$
- $t = 60.0 \text{ s}$

위치 갱신 600 번

표적마다 · 위경도로 기록

① 표적 구조체를 정의한다

표적 최대 10개 · 표적마다 기동 최대 30개
초기 위치는 위경도·고도, 속도는 동체 속력 하나로

② 위치·속도는 ECEF 에서 갱신한다

동체 속력을 ECEF 성분으로 바꾸는 방법도 고민 (추가 항목)

③ 60초 동안 0.1초 간격으로

플랫폼 하나 (정지) · 대함 표적 · 대공 표적

④ 위경도로 출력하고 그림으로

플랫폼과 표적의 궤적을 지도 위에

한 줄로 : 표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라

표적마다 600개의 점이 찍힌다. 지난 과제의 좌표변환 함수는 그대로 쓰고, 그 위에 시간과 기동을 얹는다.

00. 과제 2와 무엇이 다른가

한 점을 한 번 변환하던 일이, 매 0.1초 600번 반복하는 일이 된다

과제 2 · 한 점을 한 번 변환



입력이 한 번 들어오고, 변환 체인을 한 번 지나, 점 하나가 나온다.
시간이라는 축이 없다.

과제 3 · 매 0.1 초마다, 60 초 동안 반복



초기값을 한 번 넣고, 같은 계산을 600 번 돌린다. 매번 점이 하나씩 찍혀 선이 된다.
새로 들어온 것은 "시간" 이다.

같은 것 — 좌표변환 함수 넷은 그대로

LLA ↔ ECEF, 동체 → NED, NED → ECEF. 제공된 참고 코드에 그대로 있다.
안테나 좌표계만 이번엔 등장하지 않는다 (재는 쪽이 아니라 만드는 쪽).

새로운 것 — 속도 · 시간 · 기동

속도 : 화살표는 위치와 다르게 변환한다 (회전만)

시간 : 새 위치 = 지금 위치 + 속도 × 0.1 s. 기동 : 몇 g 로 어느 축을 도는가

과제 2의 변환 체인을 재료로 쓰고, 그 위에 시간 루프와 기동 모델을 얹는다

표적을 움직이는 원리

- 1 이번에 쓰는 좌표계 넷 — LLA · ECEF · NED · 동체
- 2 위치와 속도는 다르게 변환한다
- 3 동체 속도를 ECEF 속도로 — 회전 두 번
- 4 시간 적분 — 새 위치 = 지금 위치 + 속도 $\times$ 0.1 초
- 5 지구는 둥글다 — 60초 직진하면 11 m 떠오른다
- 6 한 스텝의 순서 — 다섯 단계

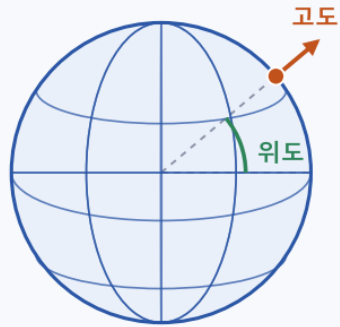
이 발표의 약속 : 각도는 도(°) 로 말하고 프로그램 안에서는 라디안. 거리는 m, 시간은 s.

01. 이번에 쓰는 좌표계 넷

입력·출력은 위경도, 계산은 ECEF, 수평면은 NED, 속력은 동체

LLA 위도 · 경도 · 고도

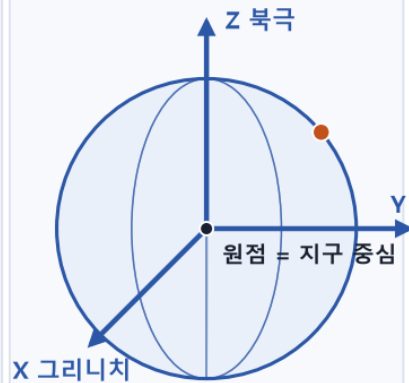
"지도 주소" — 사람이 읽는 자리



경도는 자오선에서 잰 각
고도는 타원체 면에서 잰 높이

ECEF 지구 중심 xyz

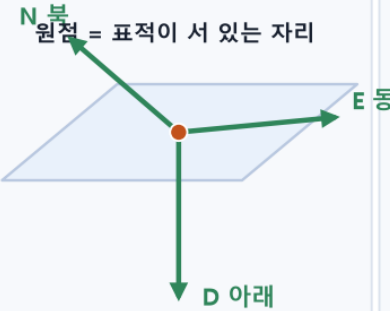
"지구에 박힌 자" — 계산하는 자리



지구와 함께 도는 직교좌표 (m)

NED 북 · 동 · 아래

"내 발밑 나침반" — 수평면 기준



표적이 움직이면 같이 따라간다

동체 앞 · 오른쪽 · 아래

"표적의 몸" — 속도가 적힌 자리



속력 V 는 늘 x 축 위에만 있다

이번 과제에서의 역할

LLA

초기값을 받고, 매 스텝 결과를 적는 형식

ECEF

위치와 속도를 600번 갱신하는 자리

NED

표적 자리의 수평면. 자세각의 기준이자 속도 방향을 만드는 자리

동체

속력 V 가 적힌 자리. 앞으로 V , 옆·아래는 0

원점이 표적 자신인 NED 가 하나 더 필요하다 — 자세각과 수평면은 표적이 서 있는 자리 기준이기 때문

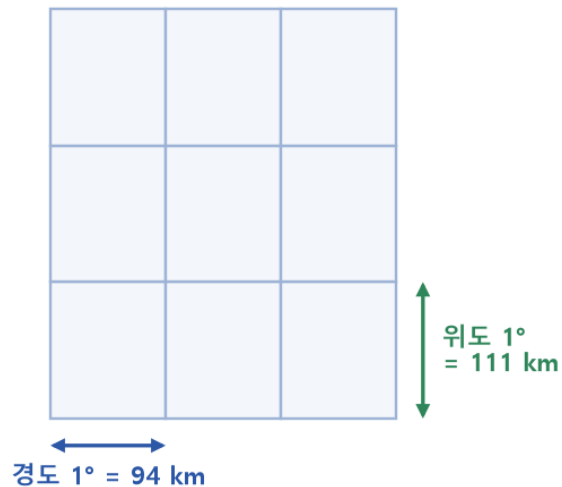
지난 과제의 NED 는 플랫폼 자리였다. 이번 NED 는 표적이 움직이면 같이 따라간다.

01. 왜 ECEF 에서 움직이나

위경도는 각도라 더할 수 없고, ECEF 는 미터라 더하면 끝난다

위경도 격자 (북위 32° 부근)

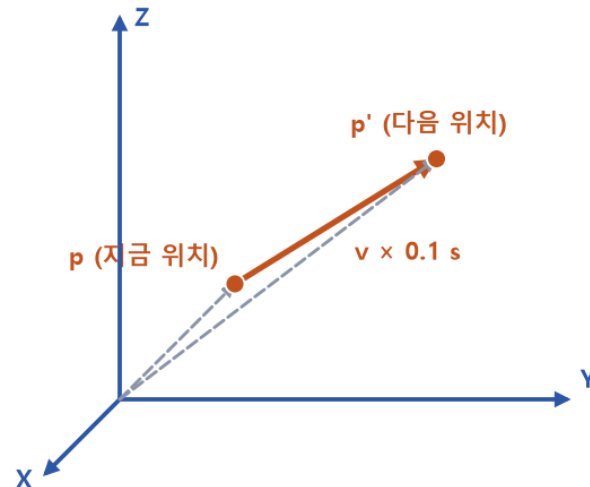
"1도" 라는 같은 눈금인데 실제 길이가 다르다



✗ 위도 + 속도 × 시간 ? 각도에 미터를 더할 수 없다

ECEF 직교좌표 (미터)

세 축이 모두 미터라 벡터를 그대로 더한다



✓ $p' = p + v \times \Delta t$ 더하기 한 줄이면 끝

① 위경도는 각도다

위도 1° 는 111 km, 경도 1° 는 94 km (32°N).
각도에 미터를 더할 수 없다.

② ECEF 는 미터다

세 축이 전부 미터라 벡터를 그대로 더한다.
 $p' = p + v \times \Delta t$, 한 줄.

③ 플랫폼과 무관한 절대 좌표

플랫폼이 움직이거나 흔들려도 표적 궤적은 그대로.
플랫폼 기준 NED 에서 움직이면 플랫폼 흔들림이 표적에 섞인다.

흐름 : 위경도로 받는다 → ECEF 로 바꾼다 → 600번 내내 ECEF 에서 더한다 → 기록할 때만 위경도로 되돌린다

과제 요구 (2) "표적 상태는 ECEF 좌표계에서 갱신" 의 이유가 이것이다.

01. 위치와 속도는 다르게 변환한다

위치는 원점이 있어야 말이 되고, 속도는 화살표 하나면 된다

기동 판단

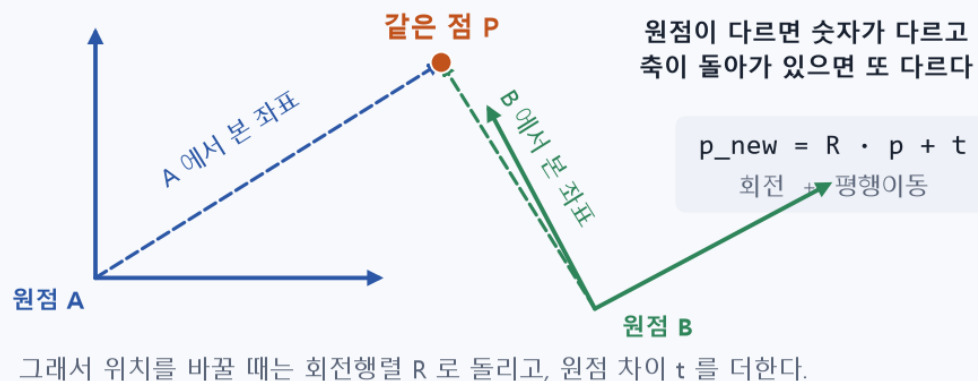
자세 갱신

속도 변환

위치 적분

LLA 변환

위치 — "어디에 있나" 는 원점이 있어야 말이 된다



속도 — "어느 쪽으로 얼마나 빨리" 는 화살표 하나면 된다
어디에 그러도 같은 화살표 v



위치 : $p_{new} = R \cdot p + t$

회전하고 원점 차이만큼 옮긴다. 지난 과제에서 계속 하던 그 일.

속도 : $v_{new} = R \cdot v$

축이 돌아간 만큼만 돌린다. t를 더하면 그 순간 속도가 아니게 된다.

과제의 "추가 고민" — 동체 속도 $V_{heading}$ 을 ECEF 성분으로 바꾸는 방법 — 의 답이 오른쪽 한 줄이다

01. 동체 속도를 ECEF 속도로 — 회전 두 번

자세각으로 한 번, 표적 자신의 위도·경도로 한 번. 평행이동은 없다

기동 판단

자세 갱신

속도 변환

위치 적분

LLA 변환

① 동체 좌표의 속도

앞으로만 간다

$$v_{body} = [V, 0, 0]$$

대공 표적 : (200, 0, 0)

자세각으로 회전
roll · pitch · yaw

회전만
✗ 위치 더하기

② NED 좌표의 속도

북·동·아래 성분으로

$$v_{ned} = C_{ned \leftarrow body} \cdot v_{body}$$

(-200, 0, 0) 남쪽으로 200

위도·경도로 회전
표적 자신의 자리 기준

회전만
✗ 위치 더하기

③ ECEF 좌표의 속도

지구 중심 축 성분으로

$$v_{ecef} = C_{ecef \leftarrow ned} \cdot v_{ned}$$

(-62.5, 86.0, -169.4) 크기 200.0

속도는 화살표라서 두 번 다 회전만 한다. ③ 에서 위도·경도는 플랫폼이 아니라 표적이 지금 있는 자리의 값이다.

① 동체 : [V, 0, 0]

표적은 앞으로만 간다.

속력 하나만 받아도 벡터가 정해지는 이유.

② 자세각으로 회전

과제 2의 $C_{ned \leftarrow body}$ (roll · pitch · yaw) 그대로.
yaw 180° → 북쪽 성분이 -200, 즉 남쪽으로 200.

③ 위도·경도로 회전

과제 2의 $C_{ecef \leftarrow ned}$ 그대로. 단, 위경도는 표적 자신의 것.

참고 함수는 플랫폼 위치를 더하므로 그 인자에 0 을 넣는다.

검산 : 결과 벡터의 크기는 항상 V 와 같아야 한다 → 200.000 m/s. 플랫폼 위치를 잘못 더하면 6,372 km/s 가 나온다

01. 시간 적분 — 새 위치 = 지금 위치 + 속도 × 0.1 초

한 스텝 안에서는 속도가 일정하다고 본다. 그래서 곱셈 한 번이면 정확하다

기동 판단

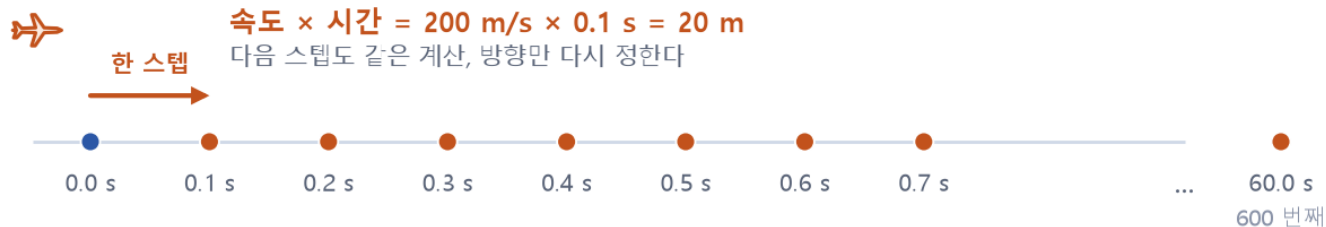
자세 갱신

속도 변환

위치 적분

LLA 변환

자동차로 치면 : 시속 720 km 로 0.1 초 가면 20 m. 그걸 600 번 더하면 12 km.
속도의 방향은 매 스텝 다시 계산하므로 (앞 장의 회전 두 번), 곡선도 이 방식으로 그려진다.



$$\text{새 위치} = \text{지금 위치} + \text{속도} \times 0.1 \text{ s}$$

$$p(t + \Delta t) = p(t) + v(t) \cdot \Delta t \quad \text{한 스텝 안에서는 속도가 일정하므로 이 한 줄로 충분하다}$$

왜 이렇게 단순해도 되나

0.1초 동안은 방향도 크기도 안 바뀐다고 본다.
곡선은 매 스텝 방향을 다시 정해서 만든다.

룬지쿠타 4차는?

가속도가 위치의 함수인 궤도 문제용.
이번엔 속도를 우리가 정해 주므로 필요 없다.

숫자 감

대공 한 스텝 20 m · 60초 12 km, 대함 한 스텝 3 m · 60초 1.8 km

$$p(t + \Delta t) = p(t) + v(t) \cdot \Delta t \quad \text{ECEF 의 } x, y, z \text{ 세 성분에 각각}$$

$\Delta t = 0.1 \text{ s}$, 600 스텝. $v(t)$ 는 직전 단계(속도 변환) 에서 매 스텝 새로 만든 값이다.

01. 지구는 둥글다 — 60초 직진하면 11 m 떠오른다

처음 속도를 그대로 쓰면 접선을 따라 지구에서 멀어진다. 매 스텝 수평면을 다시 잡아야 한다

기동 판단

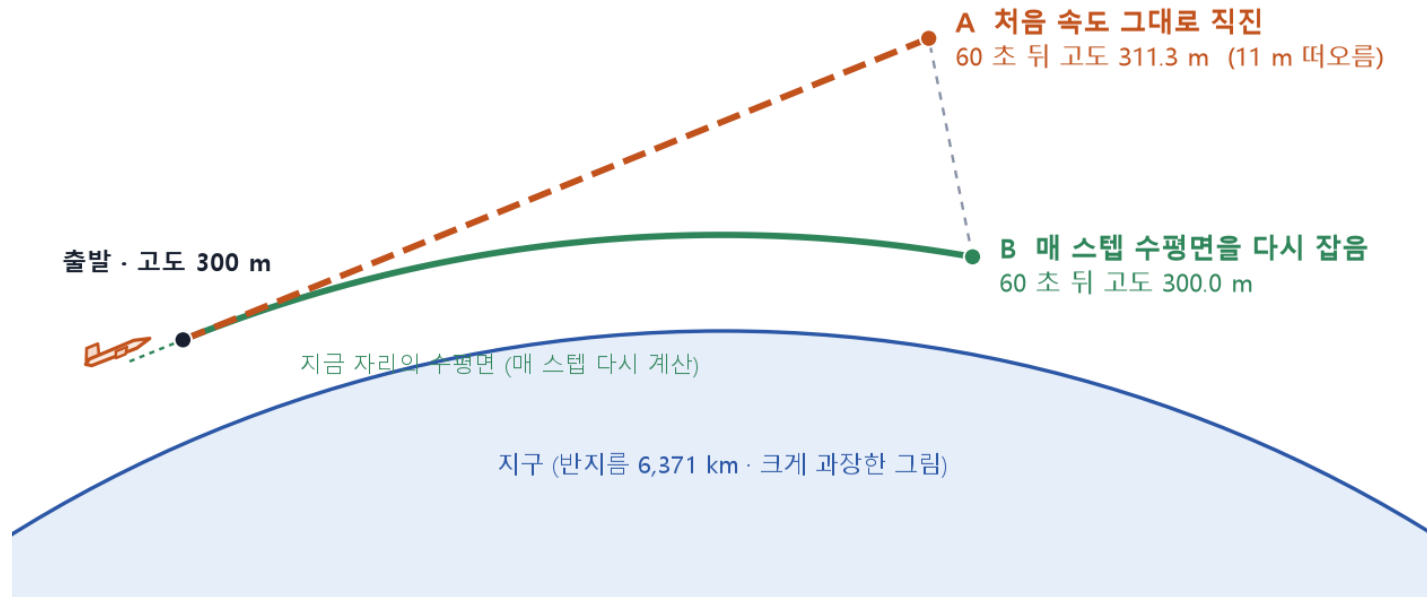
자세 갱신

속도 변환

위치 적분

LLA 변환

왜 떠오르나 : 지구가 둥글어서 접선 방향으로 12 km 가면 땅이 11 m 아래로 내려간다 ($d^2 / 2R$)
그래서 매 스텝 표적의 현재 위경도로 NED 수평면을 다시 잡고, 그 위에서 속도 방향을 다시 만든다



실험 (대공 표적 60초 직진)

60초 뒤 고도

A 처음 ECEF 속도를 60초 내내 그대로

311.3 m

B 매 스텝 현재 위경도로 NED 를 다시 잡음

300.0 m

왜 11 m 인가

접선을 따라 d 만큼 가면 땅은 $d^2 / 2R$ 만큼 내려간다.
 $12,000^2 / (2 \times 6,371,000) = 11.3 \text{ m}$. 실험값과 일치.

그래서 5단계 LLA 변환은

출력용만이 아니라, 다음 스텝의 수평면(NED 축) 을 만드는 기준이다.

위경도를 구해야 NED 축이 나오고, NED 축이 있어야 속도 방향이 나온다 — 이 순서를 지키면 수평 비행이 수평으로 유지된다

01. 한 스텝의 순서 — 다섯 단계

표적 하나가 0.1초 뒤로 가는 일. 이 순서를 600번, 표적 수만큼 반복한다

표적 하나의 한 스텝 (0.1 초)

왼쪽부터 순서대로. ① ② 는 기동이 없으면 건너뛰고, ③ ④ ⑤ 는 매 스텝 반드시 한다.



① 에 필요한 것

기동 목록, 지금 시각 t

② 에 필요한 것

각속도 $\omega = n \cdot g / V$, Δt

③ 에 필요한 것

자세각 3개, 표적 위경도

④ 에 필요한 것

ECEF 위치·속도, $\Delta t = 0.1\text{ s}$

⑤ 에 필요한 것

ECEF → LLA 변환 함수

표적 10개 × 600 스텝 = 6,000번. 순서가 중요한 건 ③ ④ ⑤ — 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면

기동 모델

- 1 기동이란 — 하중배수 g
- 2 원운동 공식 — 각속도와 선회 반경
- 3 세 축 — Roll · Yaw · Pitch
- 4 기동 시간표 — StartTime · EndTime

약속 : 기동 중에도 속력은 바뀌지 않는다. 방향만 바뀐다.

02. 기동이란 — 하중배수 g

과제의 Gravity Value 는 "몇 g 로 도는가". 속력은 그대로, 방향만 바뀐다

기동 판단

자세 갱신

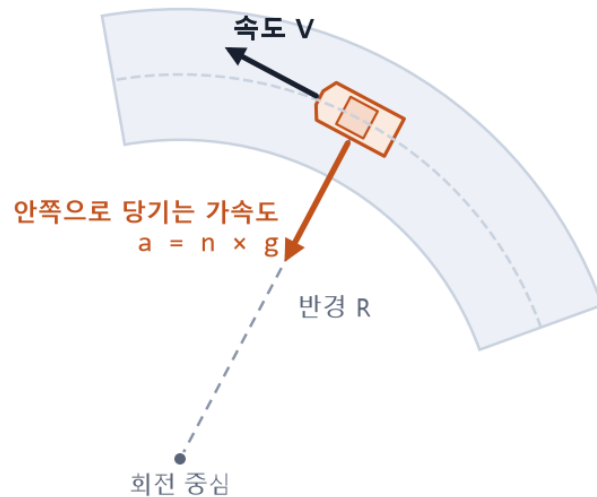
속도 변환

위치 적분

LLA 변환

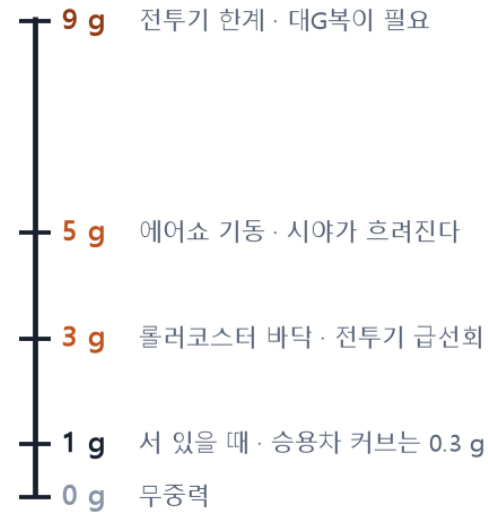
커브를 도는 자동차

속력은 그대로, 방향만 계속 바뀐다



하중배수 n ("몇 g ")

몸이 느끼는 힘이 평소 몸무게의 몇 배인가



과제의 Gravity Value 가 바로 이 n 이다

하중배수 n

안쪽으로 당기는 가속도가 중력가속도 g 의 몇 배인가.
 $a = n \times g$. 과제의 Gravity Value 가 이 n 이다.

가속도는 속도에 수직

속도 방향으로 밀지 않고 옆으로만 당긴다.
그래서 속력은 안 변하고 방향만 변한다.

숫자 감

200 m/s 표적이 3 g 로 돌면 1초에 8.4°,
30 m/s 배는 0.1 g 만 돼도 1초에 1.9°.

g 값 하나로 표적이 얼마나 빨리 도는지가 정해진다 → 다음 장의 등속 원운동 공식

02. 원운동 공식 — 각속도와 선회 반경

고등학교 물리의 등속 원운동 그대로. 같은 g 라도 느린 표적이 훨씬 빨리 돈다

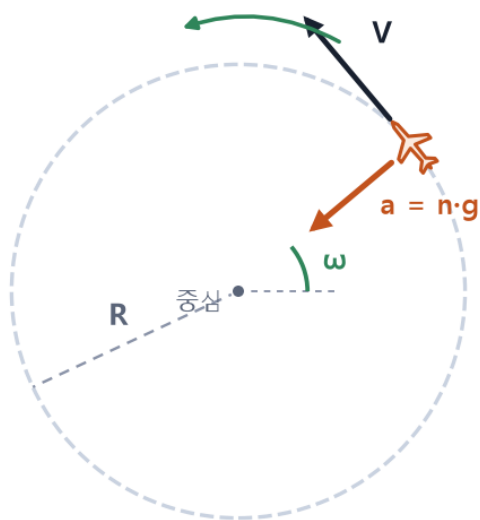
기동 판단

자세 갱신

속도 변환

위치 적분

LLA 변환



1 초에 ω 만큼 돌아간다

등속 원운동 공식

구심가속도
 $a = V^2 / R = n \cdot g$

선회 반경
 $R = V^2 / (n \cdot g)$

각속도
 $\omega = V / R = n \cdot g / V$

$g = 9.80665 \text{ m/s}^2$
 V 는 변하지 않는다

표적	하중배수	각속도	선회 반경	비고
대공 200 m/s	1 g	2.81 °/s	4,079 m	180° 에 64 s
대공 200 m/s	3 g	8.43 °/s	1,360 m	180° 에 21 s
대공 200 m/s	5 g	14.05 °/s	816 m	180° 에 13 s
대함 30 m/s	0.1 g	1.87 °/s	918 m	90° 에 48 s

각속도가 속력에 반비례한다

$\omega = n \cdot g / V$. 배는 작은 g 로도 잘 돌고, 빠른 항공기는 큰 g 가 있어야 돈다. 그래서 배 기동에는 0.1 g 같은 값을 넣어야 그럴듯하다.

프로그램에서는 한 줄 : $\omega = n \cdot g / V$ 를 구해 놓고, 매 스텝 자세각에 $\omega \times 0.1 \text{ s}$ 를 더한다 (3 g 면 한 스텝에 0.84°)

02. 세 축 — Roll · Yaw · Pitch

TurnType 은 동체의 어느 축을 돌리는가. Roll 만 속도 방향을 바꾸지 않는다

기동 판단

자세 갱신

속도 변환

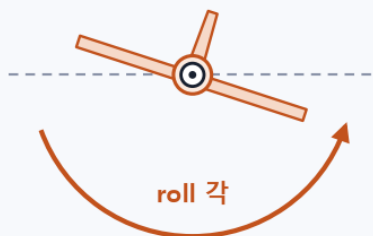
위치 적분

LLA 변환

Roll (x 축 · 앞뒤 축)

뒤에서 본 그림 · 날개가 기운다

x 축 (화면 밖으로)

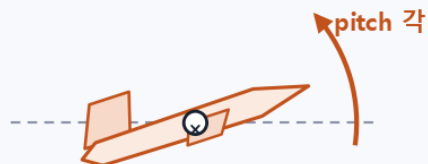


속도 방향은 그대로

Pitch (y 축 · 날개 축)

옆에서 본 그림 · 기수가 들린다

y 축 (화면 속으로)



상승 · 강하

Yaw (z 축 · 위아래 축)

위에서 본 그림 · 기수가 돌아간다

yaw 각



z 축 (화면 속으로)

좌우 선회

TurnType

축

효과

0

—

직진

1 Roll

x (앞뒤)

자세만 기움

2 Yaw

z (위아래)

좌우 선회

3 Pitch

y (날개)

상승 · 강하

Roll 은 준비 동작

롤 90° 뒤에 피치로 당기면 위가 아니라 옆으로 돈다.

실제 비행기의 선회 : 기울이고, 당긴다.

축을 돌린 뒤 속도가 어느 쪽인지는 따로 계산하지 않는다 — 자세각만 바뀌 두면 9장의 회전 두 번이 새 속도 방향을 만든다

세 축의 각속도는 모두 $\omega = n \cdot g / V$ 로 같게 둔다 (정의의 문제, 발표에서 명시).

02. 기동 시간표 — StartTime · EndTime

기동 하나 = { g, TurnType, StartTime, EndTime }. 표적마다 최대 30개

기동 판단

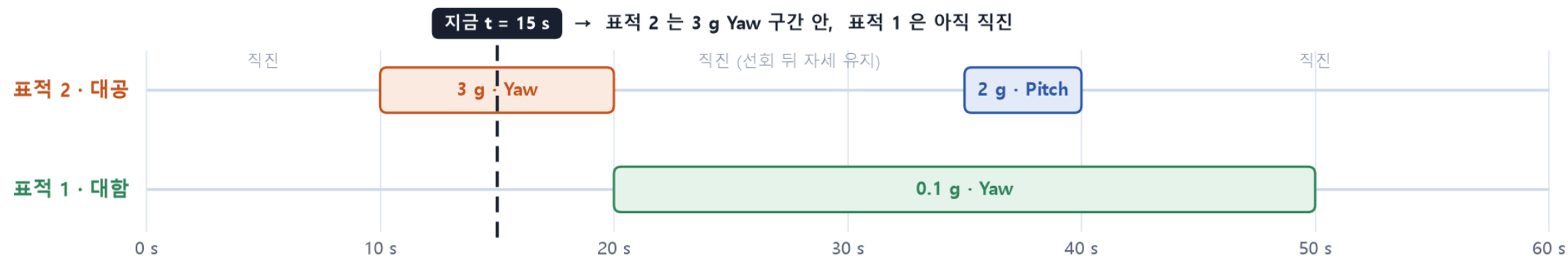
자세 갱신

속도 변환

위치 적분

LLA 변환

기동 하나 = { Gravity Value, TurnType, StartTime, EndTime } 표적마다 최대 30 개



$\text{StartTime} \leq t < \text{EndTime}$ 인 기동이 있으면 그 기동을 적용하고, 없으면 TurnType 0 (직진) 으로 본다.

구간 안이면 적용, 밖이면 직진

$\text{StartTime} \leq t < \text{EndTime}$.
없으면 TurnType 0 으로 본다.

정해 둘 규칙 셋

부호 : $n < 0$ 이면 반대 방향. 겹침 : 먼저 적힌 것.
끝난 뒤 : 그 순간 자세를 유지하고 직진.

최대 30개

배열 30칸 + 개수 하나.
과제 조건에는 기동 목록이 없다 → 예시를 우리가 정한다.

예시 기동을 넣지 않으면 직진 두 개만 그려진다 — 기동 30개짜리 구조체의 의미가 보이도록 발표용 시나리오를 하나 더 만든다

구조체와 시뮬레이션 설계

- 1 표적 구조체 — 표적 카드 한 장
- 2 시나리오와 전체 흐름
- 3 시뮬레이션 조건 — 플랫폼 하나, 표적 둘
- 4 예상 결과 — 정면 접근과 가로지르기
- 5 검증 계획 — 무엇을 확인할 것인가
- 6 결과 출력과 그림
- 7 다음 단계 — 구현 순서

과제 기타 사항 : 구조체 정의 설명 · 궤적 모의 방법 설명 · 결과 설명 · 제공 코드 이용

03. 표적 구조체 — 표적 카드 한 장

입력 칸은 사람이 적고, 상태 칸은 프로그램이 갱신한다. 구조체 모양에 과제 요구가 보이도록

표적 카드 #2

ST_Target

입력 (사람이 적는 칸)

초기 위치 위경도 · 고도로

32.12°N 126.0°E 300 m

동체 속도

200 m/s

초기 자세

roll 0° pitch 0° yaw 180°

기동 목록 (최대 30 개)

g	Type	Start	End
3	Yaw	10	20
2	Pitch	35	40
...			

초기화
한 번

상태 (프로그램이 갱신)

ECEF 위치

x, y, z [m]

ECEF 속도

vx, vy, vz [m/s]

현재 자세

roll, pitch, yaw

현재 위경도

출력용 · 매 스텝 변환



매 스텝 갱신 · 600 번
입력 칸은 건드리지 않는다

C 구조체 (ST_ 접두사 · typedef, 코딩 규칙대로)

```
typedef struct {
    FLOAT64    dGravity;        // 하중배수 [g], 부호 = 방향
    INT32       nTurnType;       // 0 없음 1 Roll 2 Yaw 3 Pitch
    FLOAT64     dStartTime;      // [s]
    FLOAT64     dEndTime;        // [s]
} ST_Maneuver;

typedef struct {
    /* 입력 - 사람이 적는 칸 */
    ST_Lla      stInitLla;        // 위도·경도·고도
    FLOAT64     dVheading;        // 동체 속도 [m/s]
    ST_Attitude stInitAtt;        // roll · pitch · yaw
    INT32       nManeuverCnt;     // <= MAX_MANEUVER (30)
    ST_Maneuver astManeuver[MAX_MANEUVER];
    /* 상태 - 프로그램이 갱신 */
    ST_Vec3     stPosEcef;        // ECEF 위치 [m]
    ST_Vec3     stVelEcef;        // ECEF 속도 [m/s]
    ST_Attitude stAtt;           // 현재 자세
    ST_Lla      stLla;           // 현재 위경도 (출력용)
} ST_Target;
```

위 다섯 줄이 입력, 아래 네 줄이 상태 — "초기값은 위경도, 갱신은 ECEF" 라는 과제 요구가 구조체 모양에 그대로 드러난다

03. 시나리오와 전체 흐름

카드가 여러 장 모이면 시나리오. 플랫폼도 속도 0 인 카드 한 장으로 둔다

프로그램 전체 흐름

플랫폼도 속도 0 인 표적 카드로 두면, 갱신 함수 하나로 전부 처리된다.



루프 안 5 단계 : 기동 판단 → 자세 갱신 → 속도 변환 → 위치 적분 → LLA 변환 (12 장)

시나리오 구조체

```
typedef struct {  
    INT32      nTargetCnt;           // ≤ MAX_TARGET (10)  
    ST_Target  astTarget[MAX_TARGET];  
    ST_Target  stPlatform;           // 속도 0, 기동 0개  
    FLOAT64    dSimTime;             // 60 s  
    FLOAT64    dDt;                  // 0.1 s  
} ST_Scenario;
```

플랫폼도 표적 카드로

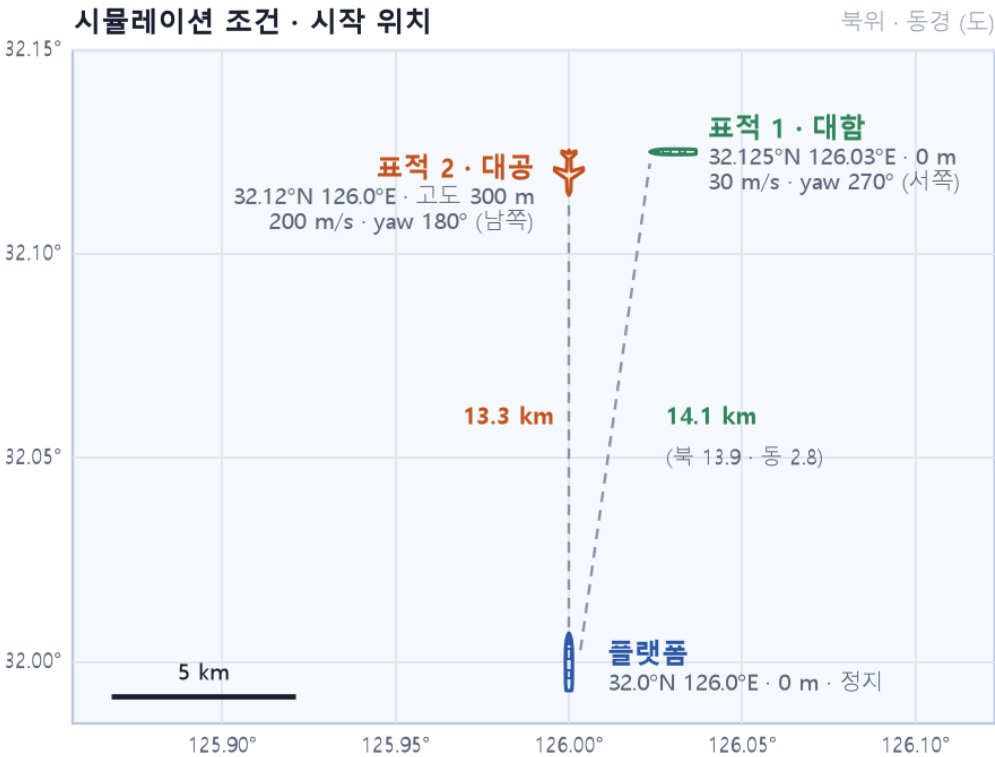
속도 0, 기동 0개인 표적일 뿐. 갱신 함수가 하나로 통일되고, 플랫폼이 움직이는 시나리오가 와도 카드 값만 바꾸면 된다.

CSV 한 줄 = 한 표적의 한 스텝

t, id, lat, lon, alt (+ 플랫폼 기준 N · E · D, 거리, 고각)
60초 × 10 Hz × 표적 수. 그림은 파이썬으로 따로 그린다.

03. 시뮬레이션 조건 — 플랫폼 하나, 표적 둘

대공 표적은 정면으로 접근하고, 대함 표적은 정북 방향을 가로지른다



항목	플랫폼	대함 표적 (1)	대공 표적 (2)
위도 · 경도	32.0°N 126.0°E	32.125°N 126.03°E	32.12°N 126.0°E
고도	0 m	0 m	300 m
속력 (동체 앞)	0 m/s	30 m/s	200 m/s
자세각 (r · y · p)	—	0° · 270° · 0° (서쪽)	0° · 180° · 0° (남쪽)
시간	60 s, 간격 0.1 s (600 스텝)		

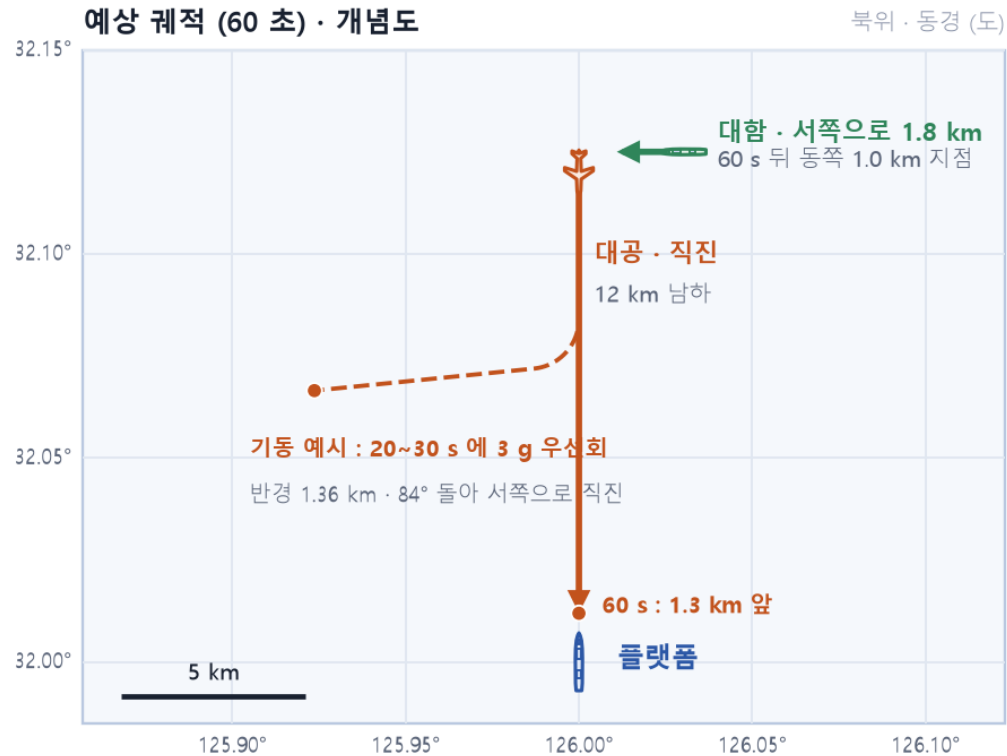
플랫폼 기준으로 바꿔 보면 (참고 코드로 계산)

대공 : 정북 13.3 km, 고각 1.2° → 정확히 우리 쪽으로 온다 (정면 접근)
대함 : 북 13.9 km · 동 2.8 km, 거리 14.1 km → 서쪽으로 가며 정북을 가로지른다
두 표적의 출발점은 550 m · 2.8 km 차이로 아주 가깝다

같은 자리에서 출발해 하나는 남쪽으로 빠르게, 하나는 서쪽으로 천천히 — 그림에서 대비가 명확하도록 짜인 조건

03. 예상 결과 — 정면 접근과 가로지르기

코드를 돌리기 전에 결과가 어떻게 나와야 하는지 먼저 그려 둔다



대공 — 정면 접근

13.3 km → 1.3 km, 고각 1.2° → 13.4°. 60초 뒤엔 거의 머리 위.
고도는 300 m 그대로여야 한다 (11장의 곡률 처리가 맞았다면).

대함 — 가로지르기

서쪽으로 1.8 km. 대공 표적 출발점 바로 북쪽을 지나 동쪽 1.0 km 지점에서 끝.

기동 예시를 넣으면

대공 표적에 20 ~ 30 s 동안 3 g 우선회 :
10초에 84° 돌아 서쪽을 보고, 그 뒤 직진. 반경 1.36 km 원호가 그려진다.
조건대로 돌린 것과 기동을 넣은 것을 나란히 보여 준다.

결과가 어떻게 해야 하는지 미리 알아야, 나온 결과가 맞는지 판단할 수 있다 → 다음 장의 검증 계획

03. 검증 계획 — 무엇을 확인할 것인가

좌표변환은 틀려도 그럴듯한 값이 나온다 (과제 2의 교훈). 점이 600개면 눈으로는 더 못 잡는다

항목	어떻게 확인하나	기대값	참고 코드로 미리 확인한 값
① 직진 거리	속력 × 시간과 비교	$200 \times 60 = 12,000 \text{ m}$	13,307 m → 1,307 m (12,000 m)
② 속력 보존	매 스텝 $ v_{\text{ecef}} $ 와 입력 속력 비교	200.000 m/s	200.000 m/s
③ 고도 유지	수평 비행 60초 뒤 고도	$300 \text{ m} \pm 0.1$	300.02 m
④ 선회 반경	궤적에서 반경을 재서 $V^2 / (n \cdot g)$ 와 비교	3 g : 1,360 m	코드 완성 후
⑤ 위경도 왕복	LLA → ECEF → LLA 오차	1 mm 미만	0.03 mm

왜 미리 정하나

코드가 내놓을 값을 먼저 알고 있어야 코드가 맞는지 알 수 있다.

①②③⑤ 는 참고 코드만으로 이미 확인했고, ④ 는 기동 코드가 붙으면 쟁다.

한 번 틀리면 바로 드러나는 실수들

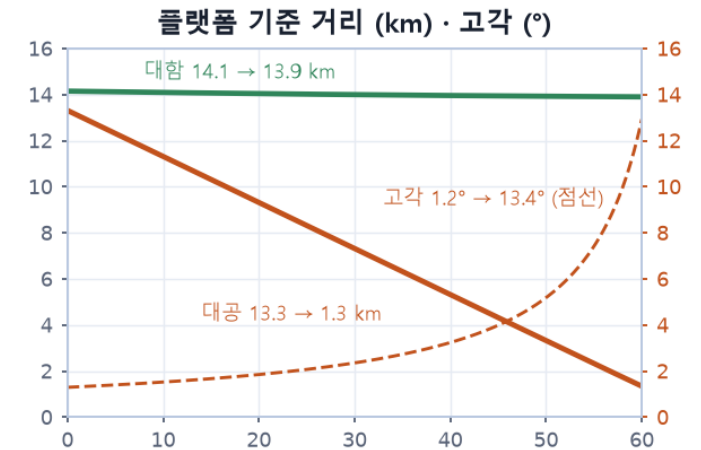
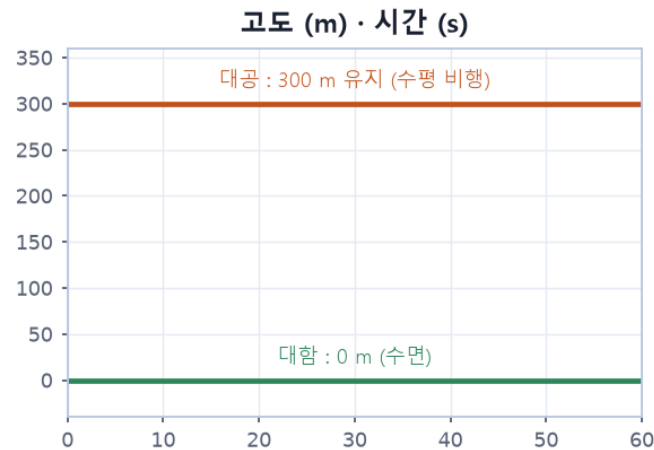
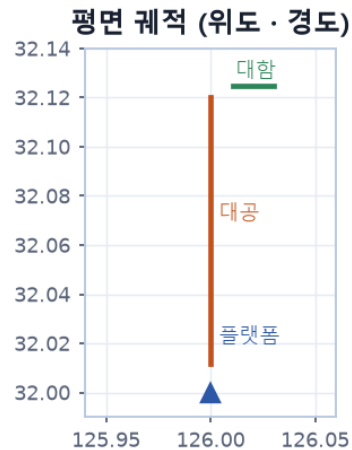
경도·위도 인자 순서 바꿈 → 북위 54° 동경 212°. 속도에 플랫폼 위치 더함 → 6,372 km/s.

수평면을 한 번만 잡음 → 고도 311 m.

이 다섯 칸이 채워지면 궤적이 "그림" 이 아니라 "결과" 가 된다

03. 결과 출력과 그림 — 위경도로 적고 세 장으로 그린다

직진만 가정하고 미리 그려 본 모양. 실제 시뮬레이션 결과로 갈아 끼울 자리



직진만 가정하고 계산한 예상 모양이다. 실제 시뮬레이션 결과로 갈아 끼울 자리.

① 평면 궤적

경도 가로, 위도 세로. 축 비율을 $1 / \cos 32^\circ$ 로 맞춰야 선이 원이 타원으로 찌그러지지 않는다.

② 고도 - 시간

대공 300 m, 대함 0 m 에서 평평해야 한다.
곡률 처리(11장) 의 검증 그림이기도 하다.

③ 거리 · 고각 - 시간

플랫폼 기준. 레이더가 실제로 보게 될 값.
대공 13.3 → 1.3 km, 고각 $1.2^\circ \rightarrow 13.4^\circ$.

CSV (t, id, lat, lon, alt, ...) → Python matplotlib → 그림 세 장. 과제 요구 (시뮬레이션 결과를 LLA 로 출력 후 그림으로 표현) 에 대응

03. 다음 단계 — 구현 순서

오늘 본 순서 그대로 코드로 옮긴다. 손계산 검산값을 먼저 만들어 둔다

- 1 구조체 셋 정의 : ST_Maneuver · ST_Target · ST_Scenario (코딩 규칙대로 ST_ 접두사, typedef)
- 2 초기화 : 위경도 → ECEF (참고 코드 f_Trans_Lla_To_Ecef, 경도가 첫째 인자)
- 3 속도 변환 함수 : 동체 → NED → ECEF, 회전 두 번, 플랫폼 위치 인자는 0
- 4 기동 판단과 자세 갱신 : 구간 검사, $\omega = n \cdot g / V$, 자세각 $+= \omega \times \Delta t$
- 5 루프와 CSV : 60 s / 0.1 s, 표적마다 5단계, 한 스텝에 한 줄
- 6 그림 세 장 : Python matplotlib, 축 비율 1 / cos 32°
- 7 검증 표 채우기 : 거리 · 속력 · 고도 · 반경 · 왕복

일정 (나흘)

1일차 수식 손계산 · 검산값
2일차 구조체 · 루프 · 속도 변환
3일차 그림 · 검증 표
4일차 발표 자료 (결과편)

손계산을 먼저 하는 이유

코드가 내놓을 값을 미리 알고 있어야
코드가 맞는지 알 수 있다 (23장).

다음 발표 : 코드와 실행 결과, 검증 표, 기동 시나리오 비교

정리

- 1 과제 3 = 과제 2의 변환 체인 + 시간 루프 + 기동 모델. 좌표변환 함수 넷은 그대로 쓴다
- 2 위치는 회전 + 평행이동, 속도는 회전만. 속도에 위치를 더하면 안 된다
- 3 새 위치 = 지금 위치 + 속도 $\times$ 0.1 초. ECEF 에서 더한다
- 4 지구는 둥글다. 매 스텝 표적 자리의 수평면을 다시 잡아야 고도가 유지된다
- 5 기동은 원운동. $\omega = n \cdot g / V$, $R = V^2 / (n \cdot g)$. 속력은 그대로
- 6 표적 카드 = 입력 칸 + 상태 칸. 플랫폼도 카드 한 장
- 7 검증 없는 궤적은 그림일 뿐. 거리 · 속력 · 고도 · 반경으로 확인한다

질문 환영합니다

다음 발표 : 코드와 실행 결과

코드 · 문서 : [radar-sw-study / Part3](https://github.com/radar-sw-study/Part3)

부록 A. 참고 코드의 함수와 함정

제공된 CoordinateTransform.c 에서 이번에 쓰는 것과, 컴파일해서 확인한 주의점

함수	이번 과제의 용도	주의
f_Trans_Lla_To_Ecef (Lon, Lat, Alt)	초기 위치 입력	경도가 첫째 인자. 바꾸면 북위 54° 동경 212°
f_Trans_Ecef_To_Lla (X, Y, Z)	매 스텝 결과 출력 + 다음 수평면 기준	5회 고정 반복. 우리 위치에서 오차 0.03 mm
f_Trans_Body_To_Ned (... , Roll, Yaw, Pitch)	동체 속력 → NED 속도	인자 순서가 Roll, Yaw, Pitch
f_Trans_Ned_To_Ecef (... , Pf_X, Pf_Y, Pf_Z, Lat, Lon)	NED 속도 → ECEF 속도	속도에는 Pf_X·Y·Z = 0. 아니면 6,372 km/s
f_Trans_Ecef_To_Ned (...)	플랫폼 기준 거리 · 고각 (결과 그림용)	각도는 전부 라디안. DEG2RAD 매크로 직접 정의
Ant 계열 4개 · Enu 계열 2개 · PfCompensation	사용 안 함	표적을 만드는 쪽이라 안테나 좌표계가 없다

Body → NED 행렬은 $R_z(\text{yaw}) \cdot R_y(\text{pitch}) \cdot R_x(\text{roll})$ 과 소수점 6자리까지 일치 — 과제 2 의 coord_frames.c 와 같은 규약

그 밖에 : 한글 주석이 UTF-8 이라 MSVC 에서 /utf-8 옵션 필요. matrixCalcLib 은 9×9 고정 행렬 (과제 2 와 같음). 상수 G_FORCE = 9.80665 가 Define.h 에 있다.

부록 B. 용어 정리

발표에 나온 말들. 질문이 들어오면 이 표를 보면서 답한다

용어	뜻	이번 과제에서
하중배수 n (g)	안쪽으로 당기는 가속도가 중력가속도의 몇 배인가	기동의 Gravity Value. $a = n \cdot g$
각속도 ω	1초에 몇 도(라디안) 도는가	$\omega = n \cdot g / V$. 매 스텝 자세각 += $\omega \times 0.1 \text{ s}$
오일러 적분	새 값 = 지금 값 + 변화율 $\times$ 시간 간격	$p \leftarrow p + v \times \Delta t$. 스텝 안 속도 일정이라 정확
자세각 roll · pitch · yaw	동체 축이 수평면(NED)에 대해 돌아간 세 각	표적의 진행 방향. yaw 180° = 남쪽
ECEF	지구 중심 원점, 지구와 함께 도는 직교좌표 [m]	위치·속도를 갱신하는 자리
NED	한 지점의 북 · 동 · 아래 수평 좌표	표적 자신의 자리 기준. 매 스텝 다시 잡는다
타원체고	WGS-84 타원체 면에서 수직으로 잰 높이	과제의 "고도". 해발고도와 약 20 m 차이
회전행렬 R	한 좌표계의 축을 다른 좌표계 축으로 돌려 읽는 3×3	위치엔 $R \cdot p + t$, 속도엔 $R \cdot v$

이상입니다. 질문 환영합니다.

코드 · 문서 : radar-sw-study / Part3